

AppleSupport Twitter Support Agent Report

Report: AppleSupport Twitter Support Agent

0. Scope note

Built on a 100-row sample (17 usable AppleSupport-directed messages), not the full dataset — see README for why. Everything below is a real result on that slice, and I’ve tried to be honest throughout about what changes once there’s actually enough data.

1. Problem framing

Reading the data changes what “good” even means here. All 13 of AppleSupport’s public replies in this sample do the same thing: a sentence of empathy, one diagnostic question, then “DM us.” Not one of them resolves anything in public. So there’s no historical “resolution text” anywhere in this dataset to ground a final answer on — the only thing that’s actually learnable from the public timeline is how Apple *triages*, not how they *fix*.

That reframes the whole assignment a bit. “Grounded in how the brand has historically resolved similar issues” has to mean grounded in the triage pattern, because that’s all that’s observable. And “auto-handle” can’t mean “the issue gets fixed without a human” — it has to mean the agent can send the same triage reply a human would send, without someone reviewing that specific reply first. The fix still happens in DM, with a person. That’s a smaller claim than “AI resolves the ticket,” but it’s the honest one given what’s in the data, and it’s what `src/escalation.py` is actually built around.

A few things I deliberately left out:

- **DM thread modeling.** The DM content isn’t public. Anything claiming to ground a “resolution” on it would just be making it up.
- **A general sentiment model.** Profanity/intensity detection is folded straight into the escalation rules instead of a separate classifier, because the only thing it needs to answer is “hold this for review or not” — it doesn’t need a full sentiment scale to do that.
- **Multi-label intent.** Real messages are often multi-symptom (failure mode #4 below is a good example), but building a taxonomy from 17 examples and then making it multi-label felt like solving a problem I don’t have enough data to validate against yet.

2. Results vs. baselines

(n=17 — see `outputs/eval_summary.md` for the full per-class breakdown)

	Intent accuracy	Decision accuracy
System	0.82	0.88
Trivial baseline (always predict majority class)	0.29	0.59
Simple keyword baseline (2 keywords, “battery”/“update”, no priority)	0.41	0.65

The trivial baseline’s majority class is `battery_drain/escalate`, and even that only gets 0.29 intent accuracy — the traffic is genuinely varied even at $n=17$. The keyword baseline is what you’d get in ten minutes without reading the data closely: it catches battery and update complaints fine but has no idea what to do with crashes, connectivity, feature conflicts, or account issues, and its escalation rule (profanity only) misses every PII-risk case entirely.

Reply quality (4-check rubric — grounded / on-topic / not-redundant / tone-appropriate, `eval/judge.py`): 0.96/1.00 average, heuristic judge. Agreement against hand-scored human labels: grounded 1.00, `on_topic` 0.82, `not_redundant` 0.82, `tone_appropriate` 0.94. The disagreements aren’t noise — see failure mode #5, they’re actually one of the more useful things that came out of this.

3. Top 5 failure modes, with real examples

1. The rules miss battery complaints that don’t say “battery.” Tweet 119268 (“drains it down 8 fucking percent”) and 119290 (“dropped 3% life”) are obviously battery complaints to anyone reading them, but neither uses the word, so the classifier falls through to `other` on both — that’s the whole reason `battery_drain` recall sits at 0.60. A keyword list is always going to have this ceiling. An embedding or LLM-based classifier (wired up, just needs `ANTHROPIC_API_KEY`) should close most of it, but that’s a hypothesis with two data points behind it, not a tested result.

2. There’s no “praise” intent, because the taxonomy was built from complaints. Tweet 119291 (“Super help - problem solved, once again in love with Apple”) is a happy customer, not a problem. There’s nowhere for that to go, so it lands in `other`, and the escalation logic conservatively holds it for review — the system ends up apologizing to someone who just said thank you. Any taxonomy built off a support-ticket dataset is going to under-represent non-support traffic like this unless you go out of your way to sample for it, and 17 examples isn’t enough volume to do that.

3. The profanity check is a blunt on/off switch. “wtf” and “this fucking phone” trigger the exact same escalation. I actually disagreed with my own system once while labeling (tweet 119253 — see the note in `golden_eval.csv`) on whether “wtf” should really count. A graded intensity score would separate mild venting from actual abuse, but building one that’s any good needs a lot more labeled examples of each than exist here — and erring conservative in the meantime is the right default for a support brand, even if it’s a blunt instrument.

4. Multi-symptom messages get squeezed into one label. Tweet 119263 complains about both broken apps *and* frequent wifi drops. The system picks `connectivity_issue` and the reply never touches the app-breakage part. That’s a real information loss, not just a labeling quirk — worth checking, on a bigger sample, how often messages are genuinely multi-symptom before deciding whether multi-label is worth the extra evaluation complexity it’d add.

5. The judge has its own blind spots, and I only caught them because I hand-scored in parallel. It flags tweet 119326 (“Love the new update!!!!” — which is sarcasm) as tone-inappropriate purely because it pattern-matches the word “love,” when the reply’s serious tone was actually the right call. It also flags tweet 119250 as a redundant question because “iPhone6” looks like an already-given iOS version to it, which it isn’t — that’s a device model. Both show up as the reason on_topic/not_redundant agreement sits at 82% instead of 100%. An automated judge built on the same kind of shallow pattern-matching as the classifier is going to share its blind spots — which is exactly why this harness checks the judge against a human label instead of trusting its own score, and why that agreement number matters more than the composite score as this scales.

4. What’s misleading about my headline number

0.82 intent accuracy and 0.96 reply quality look good on a slide. Here’s what they’re not telling you:

It’s $n=17$. One flipped label moves accuracy by six points. This is a smoke test that the pipeline runs end to end and behaves sensibly, not evidence that the classifier generalizes — the only reason the two baselines are in here is to give the number something to mean *relatively*, since it can’t mean much on its own.

The golden set was labeled by the same person who wrote the classification rules, days apart, after reading the same 17 examples over and over while building both. That’s about the most favorable condition for “agreement” you could set up. A held-out labeler, or even the same labeler blinded to which rules came from which examples, would be a much fairer test — honestly the single biggest thing I’d fix first with more time.

The reply-quality judge and the “human” ground truth are also both me. I wrote the rubric, scored the human column, and ran the heuristic separately — but one person doing both isn’t a real inter-rater check. It does prove the harness works (it caught real disagreements, see failure mode 5), just not that the judge itself is trustworthy at any kind of scale.

Every escalate/auto-handle case here is a case AppleSupport already escalated to DM in real life anyway. So “auto-handle accuracy” is really “did we ask the right triage question before punting to DM” — it says nothing about whether the underlying issue actually got resolved, because this dataset simply can’t tell us that.

And AppleSupport’s real replies barely vary, which makes the retrieval step look easier than it’ll be on a brand whose agents write more freely. A more heterogeneous brand would be a harder, more honest test of whether grounded retrieval actually helps.

5. What I’d do with one more week

- Get real volume: a genuinely random ~5–10k row subsample of the full dataset, across 2–3 brands rather than just Apple, and rerun the exact same pipeline. It already takes a brand and a CSV path as parameters — this is a config change, not a rewrite.
- Bring in a second labeler, ideally blind to the classification rules, and actually compute inter-rater agreement (Cohen’s kappa) before trusting any accuracy number built on top of the golden set.
- Add a `praise_feedback` intent and try multi-label, then check whether the LLM classification path actually closes the paraphrase gap from failure mode 1 — right now that’s a guess, not a result.
- Replace the binary profanity trigger with something graded, validated against more examples of the “venting vs. actually abusive” line that came up while labeling.

- Try to find out if the DM handoff outcome is reachable at all — it probably isn't from public data, but without it there's a hard ceiling on how far this eval can go toward proving the system actually helps customers, versus proving it asks reasonable questions.